Strandspace Studio

Query-Triggered Semantic Recall with Compact Prompt Benchmarking

White Paper - Updated April 15, 2026

Store the pieces. Store the triggers. Store the rules that let meaning form.

Executive Summary

Strandspace Studio is a working prototype for domain-specific recall. Instead of treating every answer as a fresh retrieval problem, it stores reusable constructs that can be reactivated from partial cues. The current build centers on a generic `Subjectspace` memory layer and a seeded `Soundspace` domain layer for music engineering.

The prototype is designed around a simple claim: repeated work should become cheaper after the system has already learned the right construct. Local Strandspace recall should answer stable, repeated prompts faster than an LLM round-trip, while the LLM path should be reserved for validation, expansion, or gaps that the local field cannot settle on confidently.

As of April 15, 2026, the current implementation shows three concrete results:

- The automated regression suite passes 11 of 11 tests and now reflects the active product rather than an older domain-specific prototype.
- The benchmark path now compacts the user prompt before timing the LLM route, but only when the shorter prompt can be proven to recall the same construct locally.
- Live runs continue to show the same directional outcome: local Strandspace recall is substantially faster than the LLM assist round-trip on repeated prompts, while compact prompting lowers the user-query token burden further.

Abstract

Strandspace Studio is a query-triggered semantic recall framework for reusable local memory. A query is treated as an activation event rather than a generic text search. Trigger strands identify the task, anchor strands narrow the domain, composite constructs carry reusable working knowledge, and the expression field emits a temporary answer only when a stable construct is available.

The current product includes two implementation surfaces. `Subjectspace` is the generic, user-adaptable memory system where a person can teach any subject by storing a construct label, target, objective, context, steps, notes, and tags. `Soundspace` is a seeded application layer that demonstrates the same pattern in live sound and music-engineering setups. When local evidence is strong enough, Strandspace answers directly. When the match is narrow or incomplete, the system can route to an OpenAI assist path for validation or expansion and then learn the refined result back into local memory.

This paper replaces the earlier proof-of-concept description. It reflects the current architecture, the current test suite, and the current benchmark path now visible in the Strandspace Studio interface.

1. Problem Statement

Most AI assistants still solve repeated domain questions by paying the same broad retrieval cost over and over. They search flat text, large prompt payloads, or general embeddings each time, even when the user keeps asking about the same underlying setup.

That approach is flexible, but it has three recurring drawbacks:

- It does not preserve reusable working structure well.
- It often re-sends more context than the current question actually needs.
- It makes repeated domain recall slower and more expensive than it has to be.

Strandspace addresses that problem by shifting from "answer everything from scratch" toward "reactivate a learned construct when the evidence is strong enough." The architecture does not claim to replace LLMs. It claims that a local semantic memory layer can carry the stable parts of repeated work, while the LLM path remains available for uncertainty, validation, or expansion.

2. Architectural Model

The current white paper inherits the core architecture from the Strandspace draft:

- **Trigger strands** identify intent such as recall, compare, or assist.
- **Anchor strands** narrow the active region of meaning, for example subject, target, room, console, camera, or event context.
- **Composite constructs** stabilize reusable knowledge such as a vocal setup, interview lighting setup, or host-forward music bingo scene.
- **Expression field** is the temporary workspace where the current answer forms.
- **Stabilized memory** is the learned local construct that can be reactivated later with lower effort.

This architecture is intentionally compositional. The system does not store every possible phrasing of an answer. It stores a reusable construct and the cues that can reactivate it.

3. Current Product Architecture

3.1 Subjectspace

`Subjectspace` is the generic layer intended to adapt to any subject the user wants to teach. A stored construct currently includes:

- Subject label and subject ID
- Construct label

- Target or focus
- Objective
- Structured context
- Ordered working steps
- Notes
- Tags
- Derived strands
- Provenance and learned count

The recall pipeline parses a prompt, scores stored constructs against the active cues, and emits a stable answer only when the winning construct crosses the readiness threshold. If the match is weaker, the system can recommend another local example first, or route to API assistance.

3.2 Soundspace

`Soundspace` is the first seeded applied domain. It demonstrates that the same memory pattern can serve music engineering and live sound work: microphone gain staging, karaoke rotation presets, host-forward music bingo scenes, and other repeatable setup patterns.

This domain matters because it is exactly the kind of work that repeats with small variations. A host mic setup, a small-room lead vocal scene, or a monitor strategy often needs refinement, not reinvention.

3.3 Routing Logic

The current local routing modes are:

- `local_recall`: the construct is stable enough to answer directly
- `api_validate`: local recall is usable, but the match is narrow or contested
- `api_expand`: the system found partial recall, but not enough to trust without help
- `teach_local`: local memory is too thin to justify expansion yet

That routing decision is important because it prevents the product from using an external API by default. The system first asks whether local memory is already sufficient.

3.4 Learn-Back Loop

When the OpenAI assist path returns a validated or expanded construct, the result can be saved back into Strandspace memory. This closes the loop:

1. Local recall attempts to answer.
2. API assist is used only when the local route says it is warranted.
3. The validated construct is optionally learned back into local memory.

4. Future prompts can recall the refined construct faster.

4. Compact Benchmark Prompting

One of the most important recent changes is the benchmark rewrite. The compare route no longer assumes that the original long user question is the fairest benchmark input for the LLM path. Instead, it now generates shorter candidate prompts and accepts one only if the system can prove locally that the shorter prompt reactivates the same construct.

That means the benchmark is no longer merely "local vs LLM on the same raw prompt." It is now:

1. Start from the user question.
2. Generate shorter recall candidates from the matched construct.
3. Re-run local recall on those shorter candidates.
4. Accept a compact benchmark prompt only if it lands on the same construct.
5. Time local recall and the LLM assist route against that compact, semantically equivalent prompt.

This change matters because it turns the benchmark into a better architectural test. It asks whether Strandspace can compress the working cue set while preserving recall identity.

5. Current Validation Status

The active automated regression suite passed **11 of 11 tests** on April 15, 2026. The paper intentionally removes older retired tests that no longer represent the application that is actually running.

The current suite covers four areas:

- **App shell and routing surface**
- Serves the Strandspace Studio interface
- Serves the standalone Soundspace interface
- Exposes seeded subject lists correctly

- **Subjectspace local memory**
- Stores and recalls a custom construct
- Routes narrow-but-ambiguous recall toward API validation
- Reports assist status correctly when the API key is unavailable
- Confirms that compact benchmark prompting still recalls the same construct while showing Strandspace faster than the mocked LLM round-trip
- Returns an OpenAI-backed draft that can be saved back into local memory

- **Soundspace seeded retrieval**
- Recalls a seeded mixer setup
- Stores a new sound construct for later recall
- Generates and stores a missing sound construct on demand

- **Database and cleanup behavior**
- Test runs use temporary SQLite databases
- Live cleanup removed stale test-created portrait-lighting subjects from the production data store
- The fallback database selector now avoids accidentally preferring backup or test databases when the main file is unavailable

6. Current Findings

6.1 Figure 1: Live UI benchmark capture

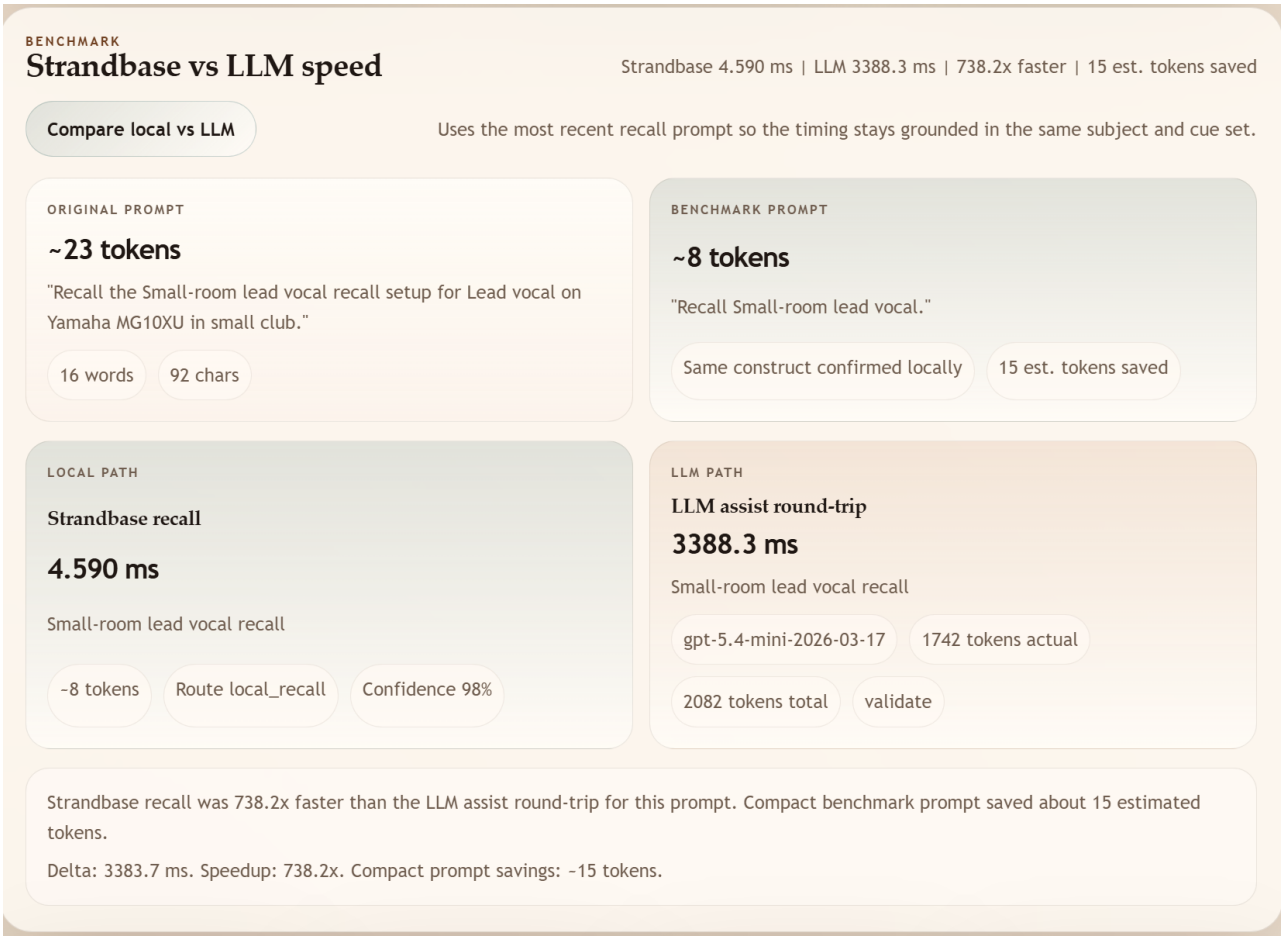

Figure 1. Strandspace Studio benchmark capture from April 15, 2026.

Figure 1 shows the current UI benchmark card after the compact-prompt rewrite. In that captured run:

- Original prompt: about 23 estimated tokens
- Benchmark prompt: about 8 estimated tokens
- Estimated prompt savings: 15 tokens
- Local Strandspace recall: 4.590 ms
- LLM assist round-trip: 3388.3 ms
- Observed speedup: 738.2x in favor of local recall

In the interface, the local path is labeled **Strandbase recall**. That label refers to the local Strandspace memory route, not to a separate architecture.

6.2 Fresh live verification after the benchmark rewrite

After the benchmark rewrite and data cleanup, a fresh live verification on the `Portrait Lighting` subject produced a second strong result:

- Original prompt: about 41 estimated tokens
- Compact benchmark prompt: about 9 estimated tokens
- Same construct confirmed locally before timing
- Estimated prompt savings: 32 tokens
- Local Strandspace recall: 0.388 ms
- LLM assist round-trip: 5462.432 ms
- Total LLM usage reported by the API: 1799 tokens

This second measurement matters because it confirms the same pattern under a different subject. The local path remains very fast once the construct is learned, and the compact benchmark prompt reduces the cue payload without losing identity.

6.3 Important interpretation note

The benchmark now reveals a useful nuance: **prompt savings and total LLM usage are not the same thing**.

The compact benchmark prompt shortens the user-query portion of the LLM request. However, the current API assist call still includes routing data and a local recall snapshot. That means the LLM can still consume a large number of total tokens even when the user prompt itself has been compressed successfully.

This is an important engineering finding, not a contradiction. It means:

- Strandspace is already reducing the user-side recall cue set.
- The current LLM assist packaging still carries more context than the minimum possible.

- A future optimization pass should reduce assist payload size further, especially when the local winning construct is already known.

7. Why These Results Matter

The current results do not claim that every question in every domain should bypass an LLM. They support a narrower and more practical conclusion:

- When the system has already learned a reusable construct, local recall is extremely cheap.
- Compact prompting can preserve construct identity with fewer user-query tokens.
- The real economic opportunity is not just answering faster, but routing less work outward.
- The product now has automated coverage around the exact flows that define this claim.

This is the strongest version of the Strandspace argument so far. The earlier prototype argued that strand-based memory could help. The current build demonstrates a working recall path, a learn-back loop, a benchmark method, and regression tests around the system boundary.

8. Limitations

The current prototype still has important limits:

- The benchmark sample size is small and prompt-specific.
- LLM timing varies with network, provider latency, and model routing.
- The current assist call still sends more context than a final optimized version should.
- Local memory quality depends on how well the user teaches the construct.
- The product is still a prototype, not a fully standardized benchmark harness.

These constraints do not weaken the core result, but they do define the next engineering work clearly.

9. Near-Term Roadmap

The next practical steps are:

- Reduce the OpenAI assist payload so that prompt compaction produces larger end-to-end token savings, not only query-side savings.
- Add benchmark history logging so repeated runs can be analyzed over time rather than one screen at a time.
- Expand the regression suite with benchmark-history and payload-size assertions.
- Continue cleaning residual legacy provenance values from stored constructs.
- Add more seeded example domains beyond music engineering and portrait lighting to demonstrate that `Subjectspace` really is subject-agnostic.

10. Conclusion

Strandspace Studio now represents a clearer and more defensible proof of concept than the earlier prototype. The active system has:

- A generic memory layer that can adapt to arbitrary subjects
- A seeded sound-engineering domain that demonstrates concrete utility
- Local-first routing with controlled API escalation
- A compact benchmark prompt selector that preserves construct identity
- An 11-test regression suite aligned with the current product
- Live evidence that repeated local recall can be dramatically faster than an LLM round-trip

The central claim of Strandspace remains the same, but it is now supported by better implementation evidence:

When meaning can be reactivated from a learned construct, the system should recall locally first and spend external tokens only when the local field is not yet sufficient.

Appendix A: Current Regression Suite Snapshot

- `GET /` serves Strandspace Studio
- `GET /soundspace` serves the standalone Soundspace app
- `GET /api/subjectspace/subjects` exposes seeded subject fields
- `POST /api/subjectspace/learn` stores and recalls a custom construct
- Subjectspace routes ambiguous recall toward API validation
- `GET /api/subjectspace/assist/status` reports disabled mode without an API key
- `POST /api/subjectspace/compare` verifies compact benchmark prompting and Strandspace speed advantage
- `POST /api/subjectspace/assist` returns an OpenAI-backed draft that can be saved
- `GET /api/soundspace` recalls a seeded mixer setup
- `POST /api/soundspace/learn` stores a new sound construct for later recall
- `POST /api/soundspace/answer` generates and stores a missing construct